Universidad Nacional de Cuyo

Facultad de Ingeniería

Licenciatura en Ciencias de la Computación

Algoritmos y Estructuras de Datos II

Wave Picking

Simulación y análisis de estrategias de preparación de pedidos en un warehouse

Informe final del proyecto semestral 2026

Docentes: Carlos Catania, Jorge Guerra

Integrantes del equipo:

Nombre y apellido	Legajo
Ailen Herrera	[COMPLETAR]
Cristian Fernandez	15136
Graciana March	[COMPLETAR]
Lucas Simionato	[COMPLETAR]

Repositorio: github.com/gracimarch/proyecto-final-aed2

Versión del documento: 09/11/2026

Índice

Resumen 5
1. Introducción 6
1.1. Contexto y problema 6
1.2. Objetivos 6
1.3. Alcance y organización del informe 6
1.4. Cómo nos organizamos 7
2. Modelo del warehouse 7
2.1. Sistema de coordenadas y grilla 7
2.2. Elementos del modelo 7
2.3. Pasillos y sentidos de circulación 8
2.4. Validaciones del modelo 9
3. Representación de datos 10
3.1. Dos representaciones complementarias 10
3.2. Construcción del grafo 10
3.3. Grafo de navegación y circulación viva 10
3.4. Estructuras auxiliares 11
3.5. Por qué lista de adyacencia y no matriz 11
4. Algoritmos 11
4.1. BFS: camino mínimo con aristas de costo 1 11
4.2. A*: la misma solución con una heurística 12
4.3. Validación de alcanzabilidad 12
4.4. Complejidad de los algoritmos principales 12
4.5. Por qué elegimos estos algoritmos 13
5. Generación de waves 13
5.1. Restricciones que debe respetar una wave 13
5.2. Las tres estrategias 14
5.3. Asignación de pickers 14
5.4. Inviabilidad y corte temprano 15
5.5. Complejidad y memoización 15
6. Ruteo de una wave 15
6.1. Paradas y celdas de recogida 15

6.2. Estrategia por orden 15
6.3. Estrategia por cercanía 16
6.4. No dejar paradas atrás en pasillos de un solo sentido 16
6.5. Descarga y zonas de despacho 16
6.6. Métricas del recorrido 16
7. Simulación y aplicación visual 17
7.1. Frames, ticks y responsabilidades 17
7.2. Colisiones, esperas y desvíos 17
7.3. Interfaz y endpoints 18
7.4. Qué se puede observar en la interfaz 18
8. Experimentación 20
8.1. Metodología 20
8.2. Distancia recorrida 21
8.3. Tiempo de ejecución y nodos expandidos 22
8.4. Estrategias de generación de waves 23
8.5. Escalado 24
8.6. Lectura de los resultados 24
9. Respuestas a las quince preguntas de la consigna 25
9.1. Pregunta 1: ¿Cómo fue modelado el Warehouse? 25
9.2. Pregunta 2: ¿Qué representación de la estructura de datos utilizaron y por qué? 25
9.3. Pregunta 3: ¿Qué algoritmos utilizaron para resolver los problemas encontrados? 26
9.4. Pregunta 4: ¿Por qué eligieron dichos algoritmos? 26
9.5. Pregunta 5: ¿Qué ocurre si todos los pasillos tienen el mismo largo? 26
9.6. Pregunta 6: ¿Cómo cambia el comportamiento del algoritmo cuando aumenta el tamaño del Warehouse? 27
9.7. Pregunta 7: ¿Cómo determinaron la distancia entre las ubicaciones de una wave? 27
9.8. Pregunta 8: ¿Cómo decidieron qué pedidos pertenecen a una misma wave? 28
9.9. Pregunta 9: ¿Qué estrategia utilizaron para ordenar las ubicaciones de una wave? 28
9.10. Pregunta 10: ¿Qué diferencias existen entre las estrategias implementadas? 28
9.11. Pregunta 11: ¿Cuál presenta menor distancia recorrida? 29
9.12. Pregunta 12: ¿Cuál presenta menor tiempo de ejecución? 29
9.13. Pregunta 13: ¿Existe una estrategia que sea mejor en todos los escenarios? 29
9.14. Pregunta 14: ¿Qué trade-offs encontraron entre tiempo de ejecución y calidad del recorrido? 30

9.15. Pregunta 15: ¿Qué modificaciones realizarían para utilizar el sistema en un Warehouse real? 30
10. Limitaciones y camino a un warehouse real 31
10.1. Limitaciones del prototipo 31
10.2. Qué cambiaríamos para un uso real 31
10.3. Alcance declarado 32
11. Conclusiones 32
11.1. Conclusiones técnicas 32
11.2. Qué nos llevamos 32
12. Referencias 33
12.1. Bibliografía y documentación 33
12.2. Uso de herramientas 33
13. Anexos 33
Anexo A. Reproducción de la aplicación y de las mediciones 33
Anexo B. Estructura del JSON de un escenario 34

Resumen

Este informe documenta el desarrollo de un simulador de Wave Picking para la preparación de pedidos en un centro de distribución. El sistema permite modelar un warehouse sobre una grilla de celdas, cargar productos y pedidos, agrupar pedidos en waves, planificar el recorrido de los pickers y simular la preparación completa con varios operarios al mismo tiempo. La implementación separa el modelo físico, la lógica de negocio, los algoritmos de grafos y la interfaz web: el backend está escrito en Python con Flask y el frontend dibuja el warehouse y la simulación sobre un canvas.

El warehouse se representa con dos estructuras complementarias. La primera es una grilla de celdas que se usa para validar el layout y dibujarlo. La segunda es un grafo dirigido con lista de adyacencia, construido a partir de los pasillos y sus sentidos de circulación, sobre el que se ejecutan las búsquedas. Para el ruteo implementamos BFS y A*, y para la generación de waves tres estrategias greedy: por pedido, por cercanía con prioridad y solo por cercanía. La simulación multi-picker resuelve colisiones, esperas y desvíos, y permite pausar, continuar, reiniciar y avanzar paso a paso.

Los experimentos se hicieron sobre los siete escenarios del repositorio y sobre cuatro escenarios generados con distinta cantidad de racks. En el escenario grande, la estrategia de cercanía redujo la distancia total de 3.751 a 2.481 pasos (−33,9 %) y el tiempo medio por wave de 4,71 ms a 2,37 ms con BFS. En el escenario de racks en esquinas, en cambio, la estrategia por orden ganó 26 a 28: no hay una estrategia que sea mejor en todos los casos. A* expandió muchos menos nodos que BFS (9.262 contra 33.969 en el escenario grande), aunque en tiempo de pared BFS resultó más rápido por el costo de mantener el heap en Python.

La conclusión principal es que las decisiones de diseño dependen del objetivo y del layout. La cercanía suele mejorar la distancia, pero puede perder contra el orden cuando la geometría tiene esquinas o pasillos de un solo sentido; BFS es una opción muy razonable para grafos de este tamaño, y A* sirve para mostrar el efecto de una heurística admisible sobre la cantidad de nodos expandidos. El informe incluye el análisis de complejidad, las mediciones, las respuestas a las quince preguntas de la consigna y una lista de las partes que el equipo debe completar antes de la entrega.

1. Introducción

1.1. Contexto y problema

Un centro de distribución recibe pedidos durante toda la jornada. Cada pedido tiene uno o más productos y cada producto está guardado en una celda de un rack, a una altura determinada. Preparar los pedidos de a uno genera recorridos largos y repetidos: el picker vuelve varias veces a la misma zona del warehouse aunque los productos estén cerca. El Wave Picking ataca ese problema agrupando pedidos en una misma wave para que un operario los recoja en una sola pasada.

El proyecto consiste en modelar ese proceso y construir una aplicación que permita simularlo. El sistema tiene que representar el warehouse, cargar productos y pedidos, generar waves respetando capacidades y prioridades, calcular recorridos que respeten los sentidos de circulación de los pasillos y mostrar la preparación con uno o varios pickers. Además, tiene que permitir comparar estrategias y medir distancia, nodos expandidos y tiempo de ejecución.

La dificultad no está en un único algoritmo, sino en la combinación de decisiones: cómo representar el espacio, cómo construir el grafo, cómo agrupar pedidos, cómo ordenar las paradas, cómo resolver colisiones y cómo medir para comparar. La consigna aclara que no se busca el óptimo global, sino una estrategia determinista, bien definida y justificada con las estructuras y algoritmos de la materia.

1.2. Objetivos

- Modelar el warehouse con una estructura de datos propia, validando dimensiones, solapamientos, racks, puertas, zonas de despacho y pasillos.
- Construir un grafo dirigido que represente la circulación real, incluyendo pasillos de sentido único y celdas sin sentido declarado.
- Implementar al menos dos estrategias de recorrido y dos buscadores de caminos mínimos, con métricas comparables.
- Generar waves de forma determinista, considerando prioridad, capacidad, peso y cercanía entre ubicaciones.
- Simular la preparación con varios pickers, respetando la regla de no ocupar la misma celda ni cruzarse.
- Construir una aplicación visual que permita cargar escenarios, generar waves, comparar estrategias y ver la simulación paso a paso.
- Analizar complejidad y comportamiento experimental en distintos tamaños de warehouse y cantidades de pedidos.

1.3. Alcance y organización del informe

El trabajo se limita a un prototipo académico. No incluye integración con un WMS real, ni tiempos estocásticos de recolección, ni optimización matemática global. Las estrategias son greedy y deterministas: no vuelven atrás para corregir una decisión anterior. Eso hace que el comportamiento sea fácil de explicar y de reproducir, a costa de perder calidad frente a una búsqueda exhaustiva.

El informe está organizado en tres bloques. Primero se describe el modelo y las estructuras de datos (secciones 2 y 3). Después se explican los algoritmos de waves, ruteo y simulación (secciones 4 a 7). Finalmente se presentan los experimentos, las respuestas a las quince preguntas de la consigna, las

limitaciones y las conclusiones (secciones 8 a 11). Los anexos incluyen el detalle de reproducción, los endpoints y la lista de partes a completar.

1.4. Cómo nos organizamos

El repositorio se trabajó con ramas por funcionalidad y merges a main. Esa dinámica nos sirvió para separar el modelado, la generación de escenarios, el ruteo, la simulación y la interfaz, y para revisar los cambios antes de integrarlos. También nos obligó a mantener los tests al día: al momento de escribir este informe la suite tiene 110 tests y 61 subtests, todos pasando.

Nos dividimos las tareas regularmente a través del sitio Trello, donde colocábamos los puntos a desarrollar y los errores que encontrábamos mientras hacíamos las pruebas. Allí, cada uno podía elegir en qué iba a empezar a trabajar, para que todo el equipo supiera y que no hubiera conflictos en el código.

2. Modelo del warehouse

2.1. Sistema de coordenadas y grilla

El warehouse se representa como un rectángulo sobre un eje cartesiano. La esquina inferior izquierda es (0, 0) y la superior derecha es (ancho_x, largo_y). El espacio se discretiza en celdas unitarias: la celda (x, y) es el cuadrado cuya esquina inferior izquierda está en (x, y), con $0 \le x < \text{ancho_x}$ y $0 \le y < \text{largo_y}$. Todos los elementos —racks, puertas, zonas de despacho, pasillos, productos y pickers— se ubican dentro de esas dimensiones.

Cada elemento rectangular se define por su ubicación (la esquina inferior izquierda), su ancho y su alto. Un rectángulo con ubicación (x, y), ancho a y alto h ocupa las celdas desde (x, y) hasta $(x + a - 1, y + h - 1)$. Esta convención es la misma en el modelo, en el generador de escenarios, en el grafo y en el dibujo del frontend, así que no hay conversiones de coordenadas entre capas.

2.2. Elementos del modelo

La clase Warehouse concentra el estado físico y el catálogo de productos. Cada elemento tiene su propia clase, con validaciones al agregarse. La Tabla 1 resume las responsabilidades.

Tabla 1. Elementos principales del modelo y su función.

Elemento	Representación	Responsabilidad
Warehouse	Dimensiones, listas y diccionarios	Contiene racks, puertas, zonas, pasillos, pickers y productos; valida solapamientos.
Rack	Rectángulo con un lado igual a 2	Estantería con celdas y alturas z; cada celda tiene una cara expuesta a un pasillo.
Door	Celda sobre el borde	Punto de acceso INBOUND, OUTBOUND o AMBOS.
DispatchZone	Rectángulo frente a una puerta	Punto de partida y llegada; tiene peso máximo y peso acumulado.
Aisle	Rectángulo con sentido	Define la dirección de circulación: N, S, E, O o AMBOS.
Picker	Posición, capacidad, carga e historial	Operario que recorre el warehouse y transporta productos.
Product	ID, nombre, ubicación (x, y, z), stock y peso	Unidad almacenada en una celda de rack.

Elemento	Representación	Responsabilidad
Order / OrderItem	Prioridad, estado e ítems	Pedido con productos, cantidades y prioridad de 1 a 5.
Wave	Pedidos, ítems, estado y pickers	Grupo de ítems que se preparan juntos en una pasada.

2.3. Pasillos y sentidos de circulación

Los pasillos son las celdas que no están ocupadas por racks, puertas ni zonas de despacho. Un pasillo declarado es un rectángulo con un sentido: N (solo hacia +y), S (solo hacia −y), E (solo hacia +x), O (solo hacia −x) o AMBOS (libre en las cuatro direcciones). Las celdas libres que no pertenecen a ningún pasillo declarado se consideran AMBOS.

La regla es más fina que “no se puede ir en contra del sentido”. En un pasillo N o S el picker puede desplazarse lateralmente sobre x para acercarse a un rack, pero no puede avanzar en contra del sentido. En un pasillo E u O pasa lo análogo sobre y. Esa distinción es importante porque permite recoger productos de ambos lados de un pasillo angosto sin habilitar giros prohibidos.

La Figura 1 muestra el escenario básico. Los racks R1, R2 y R3 están separados por pasillos verticales de sentido alternado y por bandas horizontales AMBOS. Las zonas de despacho ZA y ZB están frente a las puertas OUT e INB, y los pickers arrancan en celdas de pasillo.

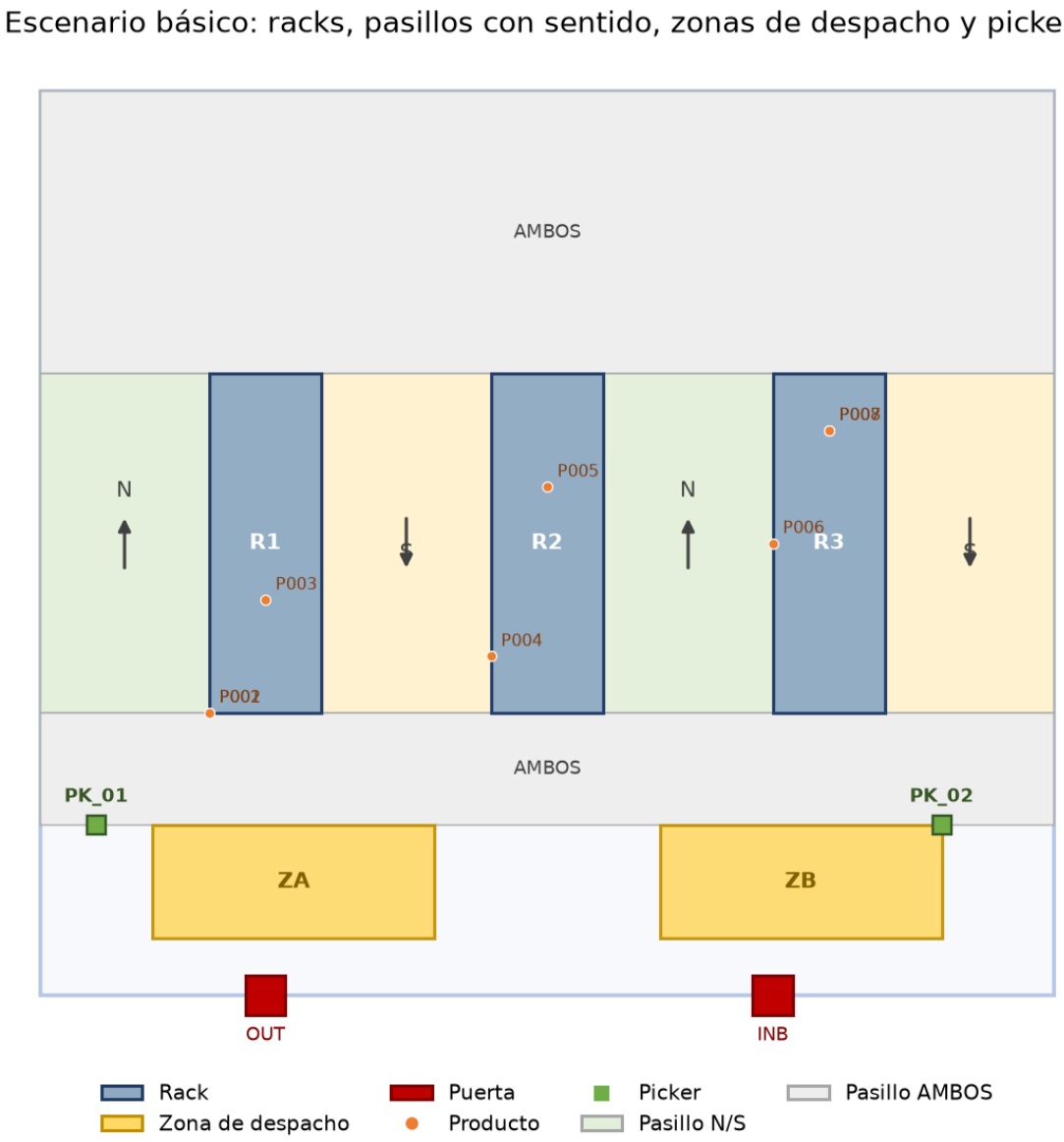


Figura 1. Escenario básico: racks, pasillos con sentido, zonas de despacho, puertas, productos y pickers.

2.4. Validaciones del modelo

El modelo no acepta un escenario inválido. Al agregar cada elemento se controla que esté dentro de las dimensiones, que no se solape con otro y que cumpla las reglas de la consigna: los racks tienen un lado igual a 2, las puertas están sobre el borde, cada puerta tiene una zona de despacho delante, los pasillos no se pisan entre sí y los pickers arrancan en celdas de pasillo. Además, al construir el WaveManager se valida que todos los productos se puedan ir a buscar y volver a la zona de despacho respetando los sentidos, y que la demanda de cada producto no supere el stock disponible.

Esas validaciones nos ahorraron varios problemas en la simulación. Un escenario que parece razonable a simple vista puede tener un pasillo de un solo sentido que entra a un callejón sin salida; si eso no se detecta antes, el picker queda encerrado y la simulación no termina. Preferimos que el sistema rechace el escenario con un mensaje concreto antes que mostrar un comportamiento raro durante la ejecución.

3. Representación de datos

3.1. Dos representaciones complementarias

El warehouse se representa de dos maneras que conviven. La primera es la grilla de celdas, que sirve para validar el layout, ubicar productos y dibujar la interfaz. La segunda es un grafo dirigido de celdas transitables, que sirve para calcular distancias y caminos. La grilla responde "¿qué hay en esta celda?"; el grafo responde "¿desde esta celda puedo ir a aquella y cuánto cuesta?". Separar las dos cosas simplificó el código: el modelo físico no sabe de BFS ni de heaps, y el grafo no sabe de racks ni de pesos.

Un nodo del grafo es una celda de pasillo, es decir, una celda que no está ocupada por un rack, una puerta o una zona de despacho. Una arista une dos celdas ortogonalmente adyacentes si el paso está permitido por los sentidos de circulación. El peso de cada arista es 1, porque cada movimiento cuesta una celda. El grafo es dirigido: en un pasillo de sentido único, la arista de ida existe pero la de vuelta no.

3.2. Construcción del grafo

La construcción recorre todas las celdas del warehouse una vez. Primero marca las celdas bloqueadas por racks, puertas y zonas, y guarda el sentido de cada celda de pasillo declarada. Después agrega un nodo por cada celda libre y, para cada nodo, mira sus cuatro vecinas. El paso de A a B se permite si B es transitable, está dentro del warehouse y el movimiento no va en contra del sentido de A ni del de B. Esa doble comprobación evita, por ejemplo, entrar a un pasillo N desde arriba bajando, aunque la celda de origen sea AMBOS.

La complejidad es O(ancho_x × largo_y): cada celda se mira una vez y tiene a lo sumo cuatro vecinas. En el escenario grande, de 60 × 40 = 2.400 celdas, quedan 1.898 nodos y 5.963 aristas. La diferencia entre celdas y nodos son los racks, las puertas y las zonas de despacho, que no son transitables.

3.3. Grafo de navegación y circulación viva

El grafo completo puede tener regiones de las que se puede entrar pero no salir. Un pasillo de sentido único que termina en un callejón sin salida es el caso típico: el picker entra, llega al fondo y no tiene aristas para volver. Para que la simulación no se trabe, el WaveManager no usa el grafo completo sino un subgrafo de navegación.

El cálculo tiene tres pasos. Primero se identifican las celdas iniciales de los pickers y se calcula la componente fuertemente conexa (SCC) que contiene a la mayor cantidad de pickers; esa es la circulación principal o "núcleo". Una SCC se obtiene con la intersección entre lo que se alcanza desde un nodo y lo que puede volver a él; en el código se hace con un recorrido en el grafo normal y otro en el grafo invertido. Segundo, se calculan todas las celdas desde las que se puede volver al núcleo, recorriendo el grafo invertido desde el núcleo. Tercero, se conservan las aristas dentro de esa circulación viva y se cortan las entradas a los callejones sin salida. Las islas donde arranquen pickers se conservan desconectadas para que esos pickers sigan teniendo un nodo válido, pero quedan marcadas como no operativas.

En el escenario grande, el grafo completo tiene 1.898 nodos y 5.963 aristas; el de navegación queda con 1.857 nodos y 5.862 aristas. Se pierden 41 nodos y 101 aristas, justamente las regiones sin retorno. El costo de construir el subgrafo es mayor que el del grafo completo (34,98 ms contra 11,15 ms en el escenario grande) porque hay que recorrer el grafo varias veces, pero se paga una sola vez por escenario.

3.4. Estructuras auxiliares

Además del grafo, el sistema usa varias estructuras para no repetir trabajo. La Tabla 2 las resume. La más importante es la tabla de distancias del WaveManager: como el grafo no cambia, la distancia desde las celdas de recogida de un rack hasta cualquier celda es siempre la misma. La primera vez que se necesita un rack se corre un BFS desde sus celdas de recogida y se guarda la fila; las siguientes veces se lee. En el escenario grande, la tabla termina con 68 filas y 126.276 entradas, y se corren 68 BFS en total para armar las 14 waves. Sin memoización, se correría un BFS nuevo por cada ítem agregado.

Tabla 2. Estructuras auxiliares y para qué se usan.

Estructura	Tipo	Uso	Costo
Grafo	dict de dicts	Vecinos y pesos; BFS y A*.	O(V + E) en memoria
Cola BFS	deque	Sacar el primero en O(1).	O(V) en memoria
Cola de prioridad A*	heapq	Sacar el nodo con menor f.	O(log V) por operación
Tabla de distancias	dict de dicts	Distancia de cada rack a cada celda; memoización.	O(L × (V + E)) una vez
Caché de recogida	dict	Celdas de recogida por producto.	O(1) por consulta
Tabla desde pickers	dict de dicts	Distancia del picker a cada celda para asignar waves.	O(P × (V + E)) una vez
Cola de waves	list ordenada	Waves esperando picker, por prioridad y antigüedad.	O(W log W) al ordenar

3.5. Por qué lista de adyacencia y no matriz

El grafo del warehouse es disperso: cada celda tiene a lo sumo cuatro vecinas, independientemente del tamaño del warehouse. Una matriz de adyacencia de 1.898 × 1.898 tendría unos 3,6 millones de entradas, casi todas en cero, mientras que la lista de adyacencia guarda solo las 5.963 aristas reales. Además, recorrer los vecinos de un nodo con una lista es O(grado), mientras que con una matriz hay que revisar toda la fila, O(V). Para BFS y A*, que pasan la mayor parte del tiempo recorriendo vecinos, la diferencia es grande.

La lista se implementó como un diccionario de diccionarios: `_adj[u] = {v: peso}`. El diccionario exterior permite buscar un nodo por su celda en O(1); el interior permite saber si existe una arista y cuál es su peso en O(1). Esa combinación es la que usan `has_edge`, `weight` y `neighbors`. La contra es que un diccionario tiene más sobrecarga de memoria que una lista de listas, pero con grafos de miles de nodos no fue un problema.

4. Algoritmos

4.1. BFS: camino mínimo con aristas de costo 1

Como todas las aristas cuestan 1, el camino con menos aristas es también el de menor costo. BFS explora por niveles: arranca en el origen, visita todos los vecinos a distancia 1, después los de distancia 2 y así. El primer momento en que llega a un nodo es la distancia mínima. La implementación usa una cola doble (`deque`) y un diccionario de niveles que también funciona como conjunto de visitados. La complejidad es O(V + E) en tiempo y O(V) en espacio.

BFS se usa en tres variantes. La primera es el BFS clásico desde un nodo, que devuelve el nivel de cada nodo alcanzable. La segunda es `bfs_distances`, un BFS multi-origen que arranca desde un conjunto de nodos y devuelve la distancia a la fuente más cercana; se usa para medir la distancia de una wave o de las zonas de despacho a cada celda. La tercera es `bfs_to_any`, que busca el camino hasta el destino más cercano de un conjunto y corta en cuanto lo encuentra; se usa en el ruteo, porque al picker le alcanza con llegar a cualquiera de las celdas de recogida de un producto.

4.2. A*: la misma solución con una heurística

A* es una búsqueda informada. Cada nodo tiene un costo real g (lo que costó llegar) y una estimación h (lo que falta). La cola de prioridad saca siempre el nodo con menor f = g + h. Si h nunca sobreestima el costo real, el camino encontrado es mínimo. En una grilla con movimientos de costo 1, la distancia Manhattan es admisible: el camino real nunca puede ser más corto que la diferencia de filas más la diferencia de columnas. Por eso A* devuelve la misma distancia que BFS, pero suele expandir menos nodos.

La implementación usa `heapq` y una tupla (f, orden, g, nodo). El contador “orden” desempata para que el heap nunca tenga que comparar dos nodos, que no son comparables. Cuando se busca hacia varios destinos, la heurística de un nodo es la menor Manhattan hacia alguno de los destinos. Es una estimación optimista y admisible, así que la optimalidad se mantiene.

4.3. Validación de alcanzabilidad

Antes de generar waves se comprueba que cada producto se pueda ir a buscar y traer de vuelta. Se hacen dos BFS desde las celdas de acceso de la zona de despacho: uno en el grafo normal, que devuelve las celdas a las que se llega desde la zona, y otro en el grafo invertido, que devuelve las celdas desde las que se puede volver a la zona. Una celda de recogida sirve si aparece en los dos resultados. Si algún producto no tiene ninguna celda que cumpla eso, el escenario se rechaza con un mensaje que lista los productos inalcanzables. La complejidad es O(V + E) por los dos BFS más O(1) por cada celda de recogida.

4.4. Complejidad de los algoritmos principales

La Tabla 3 resume la complejidad temporal y espacial de las piezas principales. V es la cantidad de nodos del grafo, E la cantidad de aristas, L la cantidad de racks distintos, N la cantidad de candidatos (pedidos o ítems), k la cantidad de paradas de una wave, d la cantidad de descargas y P la cantidad de pickers.

Tabla 3. Complejidad temporal y espacial de los algoritmos.

Algoritmo	Tiempo	Espacio	Observación
Construcción del grafo	O(ancho_x × largo_y)	O(V + E)	Cada celda y sus cuatro vecinas.
Grafo de navegación	O(P × (V + E))	O(V + E)	SCC de los pickers y recorrido inverso.
BFS desde un nodo	O(V + E)	O(V)	Camino mínimo con aristas unitarias.
BFS multi-origen	O(V + E)	O(V)	Distancia a la fuente más cercana.
A*	O(E log V) en el peor caso	O(V)	Con heurística admisible, camino mínimo.

Algoritmo	Tiempo	Espacio	Observación
Validación de alcanzabilidad	$O(V + E)$	$O(V)$	Dos BFS, uno en el grafo invertido.
Generación de waves	$O(L(V + E) + N(V + N))$	$O(L \times V)$	Tabla de distancias memoizada.
Ruteo de una wave	$O((k + d)(V + E))$	$O(V)$	Un tramo por parada y por descarga.
Simulación	$O(T \times P^2)$	$O(T \times P)$	T ticks; resolución de choques y desvíos.

4.5. Por qué elegimos estos algoritmos

BFS y A* son los dos algoritmos de búsqueda de caminos de la materia y encajan con el problema: el grafo es no ponderado (todas las aristas valen 1) y dirigido, y necesitamos caminos mínimos. BFS es la opción natural y muy eficiente para grafos de este tamaño; A* permite comparar el efecto de una heurística sobre la cantidad de nodos expandidos, que es una de las métricas que pide la consigna. No usamos programación lineal, solvers ni metaheurísticas porque la consigna las prohíbe y porque el objetivo es aplicar estructuras y algoritmos de grafos, no resolver un problema de optimización global.

Para las waves y el orden de las paradas usamos estrategias greedy. No garantizan el óptimo, pero son deterministas, fáciles de explicar y de reproducir, y permiten comparar alternativas con las mismas métricas. La prioridad de los pedidos se maneja con ordenamientos por tuplas, que en Python son estables y deterministas si se agrega el ID como último criterio de desempate.

5. Generación de waves

5.1. Restricciones que debe respetar una wave

Una wave agrupa ítems de uno o más pedidos para que se preparen en una misma pasada. No puede crecer sin límite: tiene un máximo de pedidos distintos, un máximo de unidades y un peso máximo. El peso máximo se toma como la capacidad del picker operativo más grande, para que siempre exista al menos un picker que pueda cargarla. Además, los pedidos con mayor prioridad deben considerarse antes que los de menor prioridad.

Un pedido puede quedar partido entre varias waves. Eso pasa cuando el pedido no entra completo en una wave vacía: se agregan los ítems que caben y el resto queda pendiente para la siguiente. Es la única situación en la que dividimos un pedido. En las estrategias por ítem la división es más frecuente, porque cada ítem se evalúa por separado; en la estrategia por pedido se intenta mantener el pedido entero.

5.2. Las tres estrategias

La Tabla 4 compara las tres estrategias implementadas. Las tres son greedy y deterministas: en cada paso eligen el mejor candidato según una clave, sin volver atrás.

Tabla 4. Estrategias de generación de waves.

Estrategia	Unidad de armado	Semilla	Criterio para agregar	Prioridad
por_pedido	Pedido completo	Pedido más prioritario	Mayor prioridad; si empatan, menor distancia; si no, ID.	Manda siempre
por_cercania	Ítem individual	Ítem del pedido más prioritario	Mayor prioridad; si empatan, menor distancia al rack; si no, ID.	Manda, la distancia desempata
solo_cercania	Ítem individual	Ítem más cercano a la zona de despacho	Siempre el ítem más cercano a la wave.	No se considera

En `por_pedido`, la wave arranca con el pedido más prioritario y después se agregan pedidos completos mientras entren. La distancia de la wave a cada celda se arma con la tabla de distancias y se actualiza con cada rack nuevo. La prioridad manda; la distancia solo desempata. Es la estrategia más conservadora y la que menos divide pedidos.

En `por_cercania`, la wave arranca con el ítem del pedido más prioritario y después se agrega, entre los ítems que entran, el de mayor prioridad y, si empatan, el más cercano a la wave. La distancia incluye la subida por altura z: un producto en un nivel más alto se considera un poco más lejos que uno en el piso de la misma celda. Esto agrupa ítems de distintos pedidos que están cerca, aunque los pedidos no entren completos.

En `solo_cercania`, la prioridad no se mira. La semilla es el ítem más cercano a las zonas de despacho y después se agrega siempre el ítem más cercano a la wave. Es la estrategia que mejor distancia obtiene en los experimentos, pero puede postergar pedidos prioritarios. Por eso la dejamos como alternativa de comparación, no como opción por defecto.

5.3. Asignación de pickers

La segunda etapa reparte las waves entre los pickers. Las waves sin picker esperan en una cola ordenada por prioridad y, si empatan, por orden de creación. Cada wave recibe, entre los pickers libres que la pueden cargar, el más cercano: el que tiene menos pasos desde su posición actual hasta la parada más cercana de la wave. Si empatan en distancia, gana el de menor capacidad, para que los pickers grandes queden disponibles para las waves pesadas. Si un picker no puede llegar a la wave, no se elige.

Cuando un picker termina su wave, queda libre en la celda donde descargó y compite por la siguiente de la cola. Si después de la asignación 1 a 1 sobran pickers libres, se usan como refuerzo en las waves con más paradas distintas. A cada wave se le manda el picker sobrante más cercano, con dos condiciones: que la wave tenga al menos un rack por picker (así cada uno tiene una parada propia) y que no se supere el máximo de tres pickers por wave. La distancia picker-wave se calcula con BFS desde la celda del picker, no con la tabla de racks, porque el grafo es dirigido y la distancia de ida no es la misma que la de vuelta.

5.4. Inviabilidad y corte temprano

No todos los pedidos se pueden procesar. El sistema detecta tres casos: un ítem pide más unidades que el máximo por wave, un ítem pesa más que la capacidad del picker más capaz, o la demanda total de un producto supera el stock disponible. En esos casos el pedido se registra en `unprocessable_orders` con un motivo y no se usa para armar waves. Si una wave ya armada pesa más que la capacidad de todos los pickers operativos, la simulación se corta antes de empezar y devuelve un mensaje concreto. Preferimos eso a esperar 5.000 ticks y reportar un falso bloqueo entre pickers.

También se controla que los pickers estén en la circulación viva. Un picker en un callejón sin retorno o en una isla desconectada no es operativo: no puede ir a buscar pedidos ni volver al núcleo. Esos pickers no cuentan para dimensionar waves ni para asignarlas, y el sistema los informa como aislados.

5.5. Complejidad y memoización

Con L racks distintos en los pedidos y N candidatos (pedidos en `por_pedido`, ítems en las otras dos), llenar la tabla de distancias cuesta como mucho L BFS, $O(L \times (V + E))$. Cada agregado actualiza la distancia de la wave en $O(V)$ si el rack es nuevo y recorre los candidatos en $O(N)$. El total es $O(L \times (V + E) + N \times (V + N))$. Antes de la memoización se corría un BFS por cada agregado, $O(N \times (V + E + N))$; la tabla redujo mucho el costo en escenarios con muchos ítems. En el escenario grande, las 14 waves se arman con 68 BFS en total, no con uno por ítem.

6. Ruteo de una wave

6.1. Paradas y celdas de recogida

Un producto está dentro de un rack y no se puede pisar. El picker lo recoge desde una celda de pasillo ortogonalmente adyacente a la celda del rack donde está el producto. Esa celda se llama celda de recogida. Como un rack tiene un lado de valor 2, cada celda del rack tiene al menos una cara expuesta a un pasillo, pero no limitamos la recogida a esa cara: miramos las cuatro vecinas y usamos las que sean nodos del grafo. La altura z no cambia la celda; solo cambia el tiempo de recogida.

Las paradas se agrupan por celda de rack. Si dos productos están en la misma celda a distintas alturas, se recogen juntos en una sola parada. Si dos racks enfrentados comparten una celda de pasillo, la estrategia de cercanía puede servir las dos paradas en la misma llegada. Cada parada guarda su celda de rack, los productos que se recogen y el conjunto de celdas de recogida.

6.2. Estrategia por orden

La estrategia `orden` visita las paradas en el orden en que aparecen los pedidos dentro de la wave. En cada paso busca el camino mínimo desde la celda actual hasta la primera parada pendiente, con BFS o A*. Es la estrategia más simple y la que mejor conserva la intención original del pedido. En un warehouse con una geometría complicada puede hacer rodeos, pero tiene una ventaja: no reordena nada y el recorrido es fácil de seguir para una persona.

El costo es una búsqueda por parada. Si la wave tiene k paradas y d descargas, el recorrido completo cuesta $O((k + d) \times (V + E))$.

6.3. Estrategia por cercanía

La estrategia `cercania` es un greedy de vecino más cercano. En cada paso busca, entre todas las paradas pendientes, la que esté más cerca de la celda actual. La implementación no hace una búsqueda por parada: hace un solo BFS (o A*) hacia todas las celdas de recogida pendientes a la vez. Como BFS avanza por niveles, el primer destino que toca es el más cercano; no hace falta comparar distancias una por una. Si varias paradas comparten la celda de llegada, se sirven todas juntas.

El greedy no garantiza el recorrido óptimo. Puede elegir una parada cercana que después obligue a un rodeo, sobre todo en pasillos de un solo sentido. Aun así, en la mayoría de los escenarios mejoró bastante la distancia respecto de `orden`, y el costo de calcularla es parecido: una búsqueda por paso, no una por parada.

6.4. No dejar paradas atrás en pasillos de un solo sentido

Un problema que apareció en las pruebas fue el de las paradas que quedan atrás. Imaginemos un pasillo de sentido N con dos paradas: una en y = 5 y otra en y = 8. Si el picker entra por y = 5 y sigue hasta y = 8, después no puede volver a y = 5 sin salir del pasillo y entrar de nuevo. El greedy puro podía elegir la parada de y = 8 primero y dejar la de y = 5 para el final, con un rodeo largo.

Para evitarlo, antes de elegir el próximo destino se descartan las celdas de recogida que obligarían a dejar atrás una parada pendiente. La comparación usa el “progreso” de la celda dentro del pasillo: en un pasillo N el progreso es y; en S es −y; en E es x; en O es −x; en AMBOS es 0. Una celda queda bloqueada si hay otra parada pendiente que solo se puede recoger desde celdas con menor progreso en el mismo pasillo. Si esa otra parada tiene alguna celda más adelante, no bloquea, porque se puede recoger después sin volver. Las celdas fuera de un pasillo declarado o en un pasillo AMBOS nunca se bloquean.

El costo de esta comprobación es $O(m^2)$ por paso, con m celdas de recogida pendientes (normalmente una o dos por parada). Es despreciable frente al BFS de cada tramo, que recorre todo el grafo. Esta regla fue uno de los últimos ajustes del proyecto y cambió el comportamiento en los escenarios con pasillos de un solo sentido: el recorrido dejó de dar vueltas innecesarias y la simulación se volvió más estable.

6.5. Descarga y zonas de despacho

Cuando el picker termina de recoger, vuelve a una zona de despacho. Las zonas no se vacían: el picker va a la zona más cercana que tenga lugar para al menos el ítem más liviano que lleva. Deja ahí los ítems que entran, en el orden en que los recogió, y si le sobra carga sigue hasta la siguiente zona más cercana con lugar, y así hasta dejar todo. Cada búsqueda es un solo BFS hacia las celdas de acceso de todas las zonas con lugar. Si ninguna zona tiene espacio, el recorrido falla con un mensaje claro.

El peso de la zona se controla con `peso_actual` y `peso_max_kg`. La simulación también puede encontrar una zona llena mientras el picker camina; en ese caso recalcula la descarga y busca otra zona. Es una de las razones por las que la simulación no es solo “dibujar el camino calculado”: el estado del warehouse cambia y el picker tiene que reaccionar.

6.6. Métricas del recorrido

Cada recorrido devuelve un `RouteResult` con las métricas que pide la consigna: distancia total, aristas recorridas, nodos visitados, nodos expandidos, tiempo de cálculo, tiempo de recogida y tiempo estimado.

La distancia total coincide con las aristas recorridas porque cada arista cuesta 1. Los nodos visitados son las celdas distintas por las que pasa el picker; los nodos expandidos son los que sacó de la cola el algoritmo de búsqueda sumando todos los tramos, y miden el costo del algoritmo, no el del recorrido. El tiempo de recogida es la pausa en cada parada: 1 tick base más un tick por cada nivel de altura del producto más alto de la parada. El tiempo estimado es la distancia más el tiempo de recogida, sin contar esperas por otros pickers.

7. Simulación y aplicación visual

7.1. Frames, ticks y responsabilidades

La simulación trabaja con ticks. En cada tick hace cinco cosas: decide quién se mueve, recoge y descarga, termina las waves completadas, arranca las waves que recibieron picker y guarda un frame con el estado del warehouse. El backend hace todo el trabajo y devuelve una lista de frames; el frontend solo los dibuja uno por uno. Esa división mantiene la lógica separada de la interfaz, como pide la consigna, y permite probar la simulación sin abrir el navegador.

Un frame incluye las posiciones de los pickers, sus cargas, los productos recogidos, las rutas, los eventos del tick y las métricas acumuladas. El frontend puede reproducir los frames a distintas velocidades, pausar, continuar, reiniciar y avanzar paso a paso. La simulación termina cuando no quedan waves por preparar o cuando se llega a `max_ticks`. Si algo impide terminar, se guarda el motivo en `error` y se devuelven los frames hasta ese momento.

7.2. Colisiones, esperas y desvíos

La regla de la consigna es estricta: dos pickers no pueden ocupar la misma celda en el mismo instante ni intercambiar sus celdas en un mismo paso. En cada tick se calcula el destino de cada picker y se revisan todos los pares. Si hay choque, espera el picker de la wave con menor prioridad; si ese ya estaba quieto, espera el otro. Para evitar que un picker pierda siempre, se suma un bonus de prioridad por esperas acumuladas y por esperas seguidas. Así un picker que lleva mucho tiempo esperando puede avanzar y romper el bloqueo.

Cuando un picker espera tres ticks seguidos, intenta un camino alternativo. El desvío se busca con BFS en una copia del grafo sin las celdas ocupadas por los demás y, si no hay camino, sin las próximas celdas que van a pisar. Si ya recogió todo, recalcula la descarga esquivando a los otros. También hay una lógica de estacionamiento para pickers libres: si un picker terminó su parte y quedó parado en una celda que otro necesita, se calcula un camino hasta una celda segura fuera de los recorridos activos. Sin eso, un picker libre podía bloquear para siempre la celda de descarga de un compañero.

La resolución de choques es la parte más delicada de la simulación. Probamos varias reglas antes de llegar a la actual: prioridad simple, retroceso por historial, apartarse a una celda adyacente, cadenas de pickers libres y desvío. La combinación que quedó es la que mejor se comportó en los escenarios con pasillos angostos y varios pickers. No es una demostración formal de ausencia de deadlocks, pero tiene salvaguardas: límite de iteraciones por tick, bonus por espera y desvío cuando el picker se traba.

7.3. Interfaz y endpoints

La aplicación es una página web servida por Flask. El frontend está hecho con HTML, CSS y JavaScript, y dibuja el warehouse sobre un canvas. El usuario puede cargar un escenario JSON, generarlo proceduralmente, editar racks y pasillos, ver productos y pedidos, generar waves, comparar estrategias y ejecutar la simulación. La Tabla 5 resume los endpoints principales.

Tabla 5. Endpoints de la API.

Método y ruta	Función	Devuelve
GET /	Página principal.	HTML del simulador.
POST /api/scenarios/generate	Genera un escenario procedural.	Warehouse, productos y pedidos.
POST /api/waves/generate	Arma waves y asigna pickers.	Waves con sus ítems y pickers.
POST /api/routes/plan	Calcula el recorrido de una wave.	Camino, paradas, descargas y métricas.
POST /api/routes/compare	Corre todas las combinaciones.	Comparación de estrategias y buscadores.
POST /api/simulacion	Simula la preparación completa.	Un frame por tick y métricas.
POST /api/escenario/validar	Valida un escenario guardado.	Aceptado o error con motivo.

El diseño es stateless: el cliente manda el estado completo del escenario en cada request y el servidor no guarda nada entre llamadas. Eso simplifica el despliegue y evita problemas de sesión, a costa de mandar más datos por request. Para el tamaño de los escenarios del proyecto no fue un problema.

7.4. Qué se puede observar en la interfaz

La visualización no muestra solo el resultado final. Muestra el warehouse con racks, pasillos y sentidos, puertas, zonas de despacho, productos y pickers; permite seleccionar una wave; dibuja el recorrido calculado; y durante la simulación muestra a los pickers moverse, los productos que se van recogiendo y los eventos de espera o desvío. También muestra las métricas de cada estrategia para comparar. La Figura 2 resume la separación de capas del sistema.

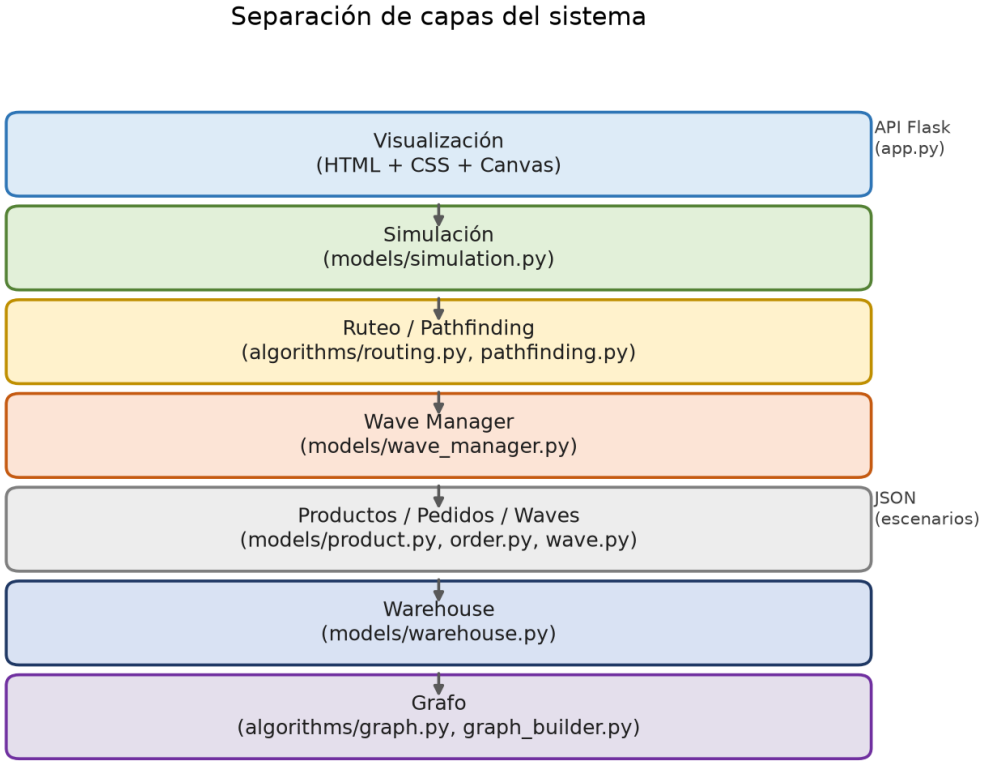


Figura 2. Separación de capas: modelo, algoritmos, simulación y visualización.

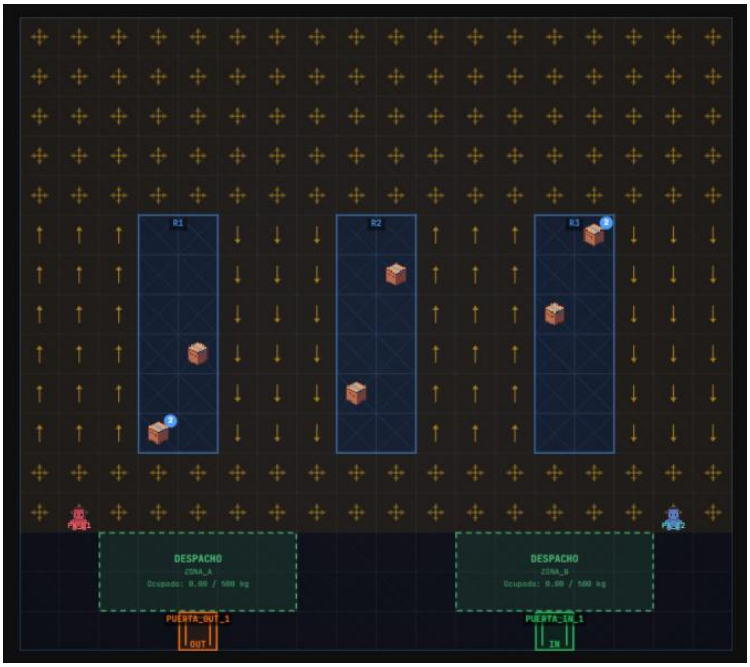


Figura 3. Vista general del warehouse con pasillos y sentidos

8. Experimentación

8.1. Metodología

Medimos sobre los siete escenarios del repositorio y sobre cuatro escenarios generados con 4, 8, 16 y 32 racks. Los escenarios del repositorio cubren casos distintos: uno básico, uno pensado para la cercanía, uno con cuello de botella, uno con varias zonas de despacho, uno con niveles z y sentidos, uno con racks en esquinas y uno grande de 60 × 40. Los generados se usaron para ver cómo crece el costo cuando aumenta la cantidad de racks y de nodos, manteniendo 30 pedidos y 2 pickers.

Cada medición se repitió cinco veces y se reporta la mediana. Para los tiempos usamos `time.perf_counter()` dentro de `plan_route`, que mide solo el cálculo del recorrido, no el dibujo ni la serialización. El tiempo de armado de waves se midió alrededor de `generate_waves`. Los tiempos de construcción del grafo se midieron por separado, con cinco repeticiones. No se hizo un calentamiento explícito de la máquina; la mediana de cinco corridas reduce el ruido, pero no lo elimina. Las comparaciones entre estrategias son válidas porque se hicieron en la misma máquina y en el mismo proceso.

Los valores de tiempo son indicativos. En Python, el costo de una operación depende mucho de la implementación y de la carga de la máquina; por eso el análisis se apoya también en los nodos expandidos, que son una medida más estable del trabajo del algoritmo. La Tabla 6 resume los escenarios y el tamaño del grafo de navegación.

Tabla 6. Escenarios utilizados y tamaño del grafo de navegación.

Escenario	Dimensiones	Celdas	Nodos	Aristas	Waves
Básico	18 × 16	288	230	692	1
Cercanía	12 × 10	120	97	310	2
Cuello de botella	14 × 12	168	129	362	2
Despacho múltiple	16 × 14	224	170	478	2
Niveles z y sentidos	18 × 16	288	231	676	2
Racks en esquinas	12 × 6	72	51	144	1
Grande	60 × 40	2.400	1.857	5.862	14

8.2. Distancia recorrida

La Tabla 7 compara la distancia total de la estrategia por orden contra la de cercanía, usando BFS en los dos casos. La distancia se mide en pasos del picker, que coinciden con las aristas recorridas. En la última fila se suma el total de los siete escenarios; no es una corrida única, sino la suma de las distancias de cada uno.

Tabla 7. Distancia total por escenario (BFS).

Escenario	Orden	Cercanía	Variación
Básico	76	52	-31,6 %
Cercanía	40	40	0,0 %
Cuello de botella	99	81	-18,2 %
Despacho múltiple	121	97	-19,8 %
Niveles z y sentidos	92	68	-26,1 %
Racks en esquinas	26	28	7,7 %
Grande	3.751	2.481	-33,9 %
Total de los siete escenarios	4.205	2.847	-32,3 %

La cercanía gana en cinco escenarios, empata en uno y pierde en uno. La mejora más grande está en el escenario grande: 3.751 contra 2.481 pasos, un 33,9 % menos. En el básico baja de 76 a 52 (−31,6 %), en cuello de botella de 99 a 81 (−18,2 %), en despacho múltiple de 121 a 97 (−19,8 %) y en niveles z de 92 a 68 (−26,1 %). En el escenario de cercanía hay empate en 40, y en el de racks en esquinas el orden gana 26 a 28: la cercanía empeora un 7,7 %.

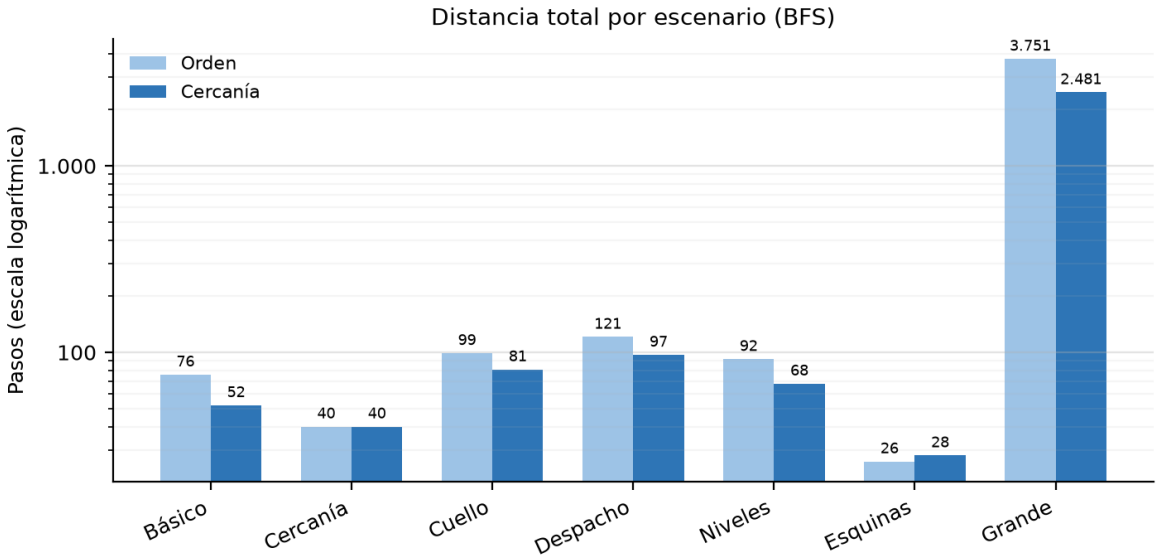


Figura 7. Distancia total por escenario con BFS. La escala es logarítmica para que se vean los escenarios chicos y el grande en la misma figura.

8.3. Tiempo de ejecución y nodos expandidos

La Tabla 8 muestra las cuatro combinaciones de estrategia y buscador en el escenario grande, que es el que más se acerca a un caso interesante. El tiempo total es la suma de los tiempos de los catorce recorridos; el tiempo medio por wave es ese total dividido por catorce. Los nodos expandidos se suman en todos los tramos de todos los recorridos.

Tabla 8. Escenario grande: distancia, tiempo y nodos expandidos por combinación.

Combinación	Distancia	Tiempo total (ms)	Tiempo medio/wave (ms)	Nodos expandidos
Orden + BFS	3.751	65,98	4,7131	71.210
Orden + A*	3.751	64,90	4,6354	18.117
Cercanía + BFS	2.481	33,22	2,3728	33.969
Cercanía + A*	2.481	57,48	4,1054	9.262

El primer resultado llamativo es que A* expande muchos menos nodos que BFS: 18.117 contra 71.210 con orden (74,6 % menos) y 9.262 contra 33.969 con cercanía (72,7 % menos). Eso confirma que la heurística de Manhattan orienta bien la búsqueda. El segundo resultado, menos intuitivo, es que en tiempo de pared BFS suele ganar: con cercanía, 2,37 ms por wave contra 4,11 ms de A*. La razón es el costo de mantener el heap y de calcular la heurística en cada vecino; en un grafo de 1.857 nodos, ese costo pesa más que el ahorro de nodos. Con orden, en cambio, A* quedó apenas por debajo de BFS (4,64 contra 4,71 ms), una diferencia que está dentro del ruido de las mediciones.

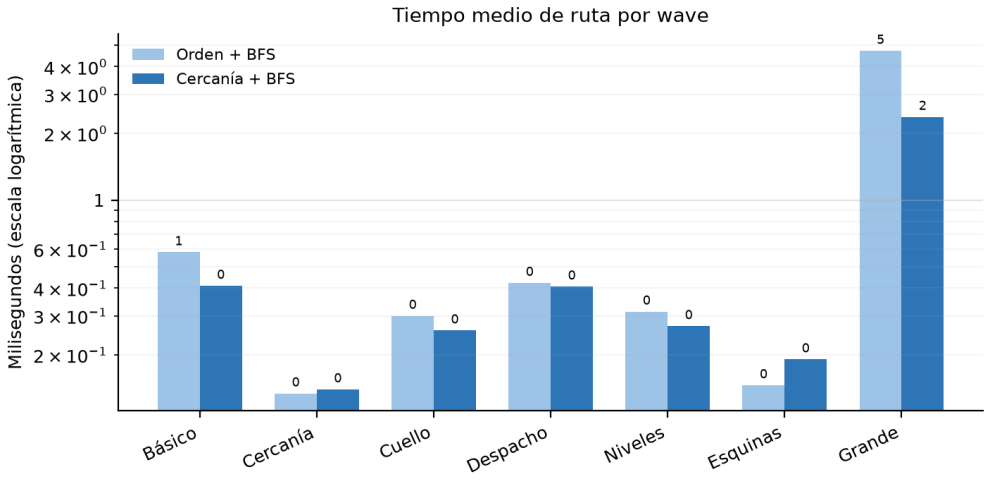


Figura 8. Tiempo medio de ruta por wave con BFS. La escala es logarítmica.

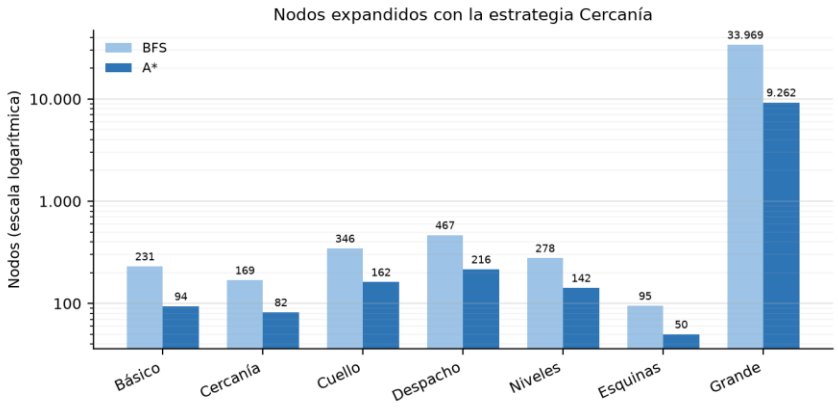


Figura 9. Nodos expandidos con la estrategia de cercanía: BFS contra A.*

8.4. Estrategias de generación de waves

La Tabla 9 compara las tres estrategias de waves en el escenario grande, siempre con cercanía + BFS para el ruteo. Las tres generan 14 waves, así que la diferencia está en qué ítems quedaron juntos y en qué orden. El tiempo de armado incluye la construcción de la tabla de distancias y la asignación de pickers; el tiempo de ruteo es la suma de los catorce recorridos.

Tabla 9. Estrategias de waves en el escenario grande (ruteo con cercanía + BFS).

Estrategia	Waves	Distancia	Armado (ms)	Ruteo (ms)	BFS
Por pedido	14	2.904	222,8	40,9	68
Por cercanía	14	2.481	228,0	34,4	68
Solo cercanía	14	2.167	235,5	30,9	69

La estrategia solo por cercanía obtiene la menor distancia: 2.167 pasos, un 25,4 % menos que por pedido (2.904) y un 12,7 % menos que por cercanía con prioridad (2.481). El costo es que ignora la prioridad: un pedido urgente puede quedar para el final si sus productos están lejos. La estrategia por cercanía con prioridad queda en el medio: mejora un 14,6 % la distancia de por pedido y sigue respetando el orden de prioridad. El tiempo de armado es apenas mayor en las estrategias por ítem (228,0 y 235,5 ms contra 222,8 ms), pero el tiempo de ruteo baja porque los recorridos son más cortos (34,4 y 30,9 ms contra 40,9 ms).

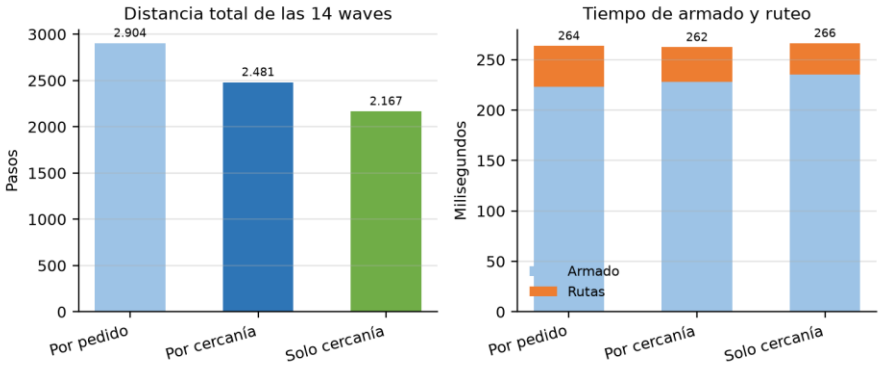


Figura 10. Estrategias de waves: distancia total y tiempo de armado más ruteo.

8.5. Escalado

La Tabla 10 muestra los escenarios generados con 4, 8, 16 y 32 racks, todos con 30 pedidos, 6 pasillos, 2 pickers y semilla 42. El objetivo no es comparar estrategias, sino ver cómo crece el costo cuando crece el warehouse. La distancia total y el tiempo de armado son de la estrategia por cercanía; el tiempo de ruta es el promedio por wave con cercanía + BFS.

Tabla 10. Escalado con escenarios generados (30 pedidos, 2 pickers, semilla 42).

Racks	Dimensiones	Nodos	Waves	Distancia	Tiempo ruta/wave (ms)	Armado (ms)
4	22 × 10	177	8	276	0,2287	12,5
8	22 × 14	247	8	388	0,4545	19,0
16	22 × 18	275	8	438	0,3939	21,0
32	22 × 26	355	8	654	0,7425	32,5

El grafo crece de 177 a 355 nodos y el tiempo medio de ruta de 0,23 a 0,74 ms. El crecimiento no es perfectamente monótono: el escenario de 16 racks tarda un poco menos por wave que el de 8 (0,39 contra 0,45 ms), probablemente porque la distribución de productos y la forma del layout cambian los recorridos. La tendencia general sí es creciente. El tiempo de armado sube de 12,5 a 32,5 ms, porque hay más racks distintos y más filas en la tabla de distancias. En el escenario grande, con 1.857 nodos, el tiempo medio por wave con cercanía + BFS es 2,37 ms, coherente con la tendencia.

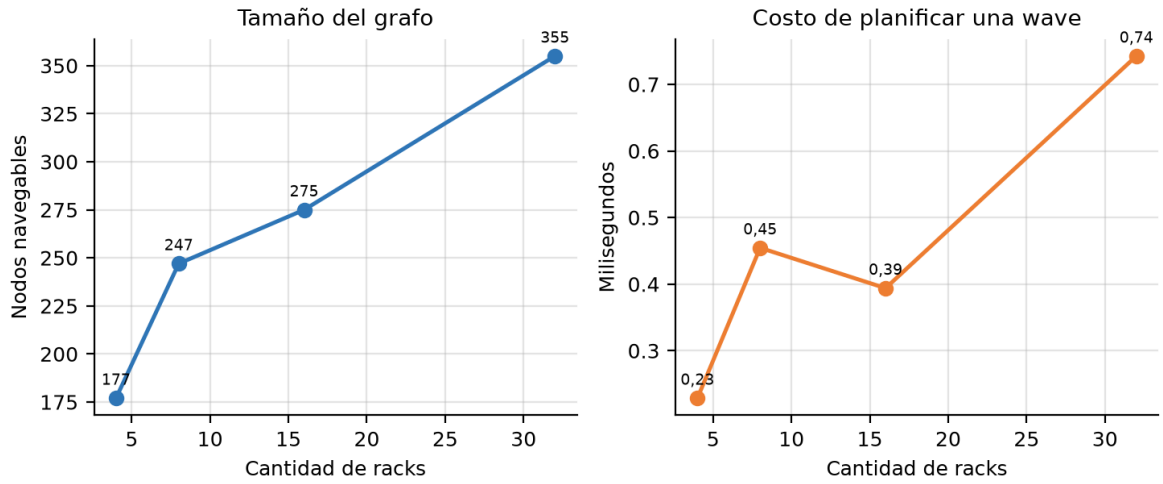


Figura 11. Escalado: tamaño del grafo y costo de planificar una wave.

8.6. Lectura de los resultados

- La cercanía mejora la distancia en la mayoría de los escenarios, pero no en todos. El contraejemplo de racks en esquinas es importante: muestra que el greedy puede empeorar cuando la geometría castiga un orden que parece cercano a simple vista.
- BFS es más rápido en tiempo de pared en casi todos los casos, aunque A* expanda muchos menos nodos. Para este tamaño de grafo, el costo del heap en Python pesa más que el ahorro de nodos. Si el grafo creciera mucho, A* probablemente se volvería más competitivo.

- La estrategia de waves importa tanto como la de ruteo. Solo cercanía reduce la distancia un 25,4 % respecto de por pedido, pero ignora la prioridad. Por cercanía con prioridad es un compromiso razonable.
- El tiempo de armado de waves es del orden de cientos de milisegundos en el escenario grande, mucho más que el ruteo. La tabla de distancias memoizada es lo que evita que ese costo se dispare.
- Los resultados son de un prototipo académico, en una máquina y con escenarios sintéticos. No deben leerse como un benchmark industrial.

9. Respuestas a las quince preguntas de la consigna

Esta sección responde, una por una, las preguntas de la sección 22 de la consigna. Las respuestas retoman lo explicado en las secciones anteriores y agregan los números de los experimentos cuando corresponde. La idea es que se puedan leer de forma independiente, sin tener que recorrer todo el informe.

9.1. Pregunta 1: ¿Cómo fue modelado el Warehouse?

El warehouse se modeló como un rectángulo sobre una grilla de celdas unitarias, con origen en (0, 0) y dimensiones (ancho_x, largo_y). Cada elemento físico es un rectángulo con ubicación, ancho y alto: racks, zonas de despacho y pasillos. Las puertas son celdas sobre el borde y los pickers son puntos con posición, capacidad e historial. Los productos se guardan dentro de los racks en una terna (x, y, z), donde z es la altura.

La clase Warehouse concentra el estado y valida cada elemento al agregarlo. No es un contenedor pasivo: controla que los racks tengan un lado igual a 2, que las puertas estén sobre el borde, que cada puerta tenga una zona de despacho delante, que no haya solapamientos y que los pasillos no se pisen. Además, mantiene índices de celdas ocupadas para que las validaciones sean O(1) por celda. La representación física se usa para validar y dibujar; para navegar se construye un grafo aparte.

9.2. Pregunta 2: ¿Qué representación de la estructura de datos utilizaron y por qué?

Usamos dos representaciones complementarias. Para el espacio físico, una grilla implícita: no guardamos una matriz de celdas, sino que cada elemento sabe qué celdas ocupa y el Warehouse mantiene diccionarios de celdas bloqueadas, celdas de pasillo y posiciones de pickers. Para la navegación, un grafo dirigido con lista de adyacencia implementada como diccionario de diccionarios: `_adj[u] = {v: peso}`.

Elegimos la lista de adyacencia porque el grafo es disperso: cada celda tiene a lo sumo cuatro vecinas, así que la cantidad de aristas es O(V), no $O(V^2)$. Una matriz de adyacencia para el escenario grande tendría 1.898 × 1.898 entradas, casi todas vacías; la lista guarda solo 5.963 aristas. Además, recorrer los vecinos de un nodo es O(grado), que es exactamente lo que necesitan BFS y A*. El diccionario exterior da acceso O(1) por celda y el interior permite consultar una arista y su peso en O(1).

También usamos estructuras auxiliares: `deque` para la cola de BFS, `heapq` para la cola de prioridad de A*, diccionarios para la tabla de distancias y listas ordenadas para la cola de waves. Todas son estructuras de la biblioteca estándar de Python, no de terceros, y se explican en el código.

9.3. Pregunta 3: ¿Qué algoritmos utilizaron para resolver los problemas encontrados?

Para construir el grafo usamos un recorrido de la grilla y la regla de circulación de la consigna. Para validar alcanzabilidad, dos BFS: uno en el grafo normal y otro en el invertido. Para calcular caminos mínimos, BFS y A* con heurística Manhattan. Para medir la distancia de una wave o de las zonas de despacho a todas las celdas, BFS multi-origen. Para detectar la circulación viva, componentes fuertemente conexas calculadas con recorridos en el grafo normal y en el invertido.

Para armar waves usamos tres estrategias greedy: por pedido, por cercanía con prioridad y solo por cercanía. Para asignar pickers, una cola por prioridad y una elección greedy del picker más cercano que pueda cargar la wave, con best fit como desempate. Para ordenar las paradas, dos estrategias: orden de los pedidos y vecino más cercano, con una regla adicional para no dejar paradas atrás en pasillos de un solo sentido. Para la simulación, un bucle por ticks con resolución de colisiones, esperas, retrocesos, estacionamiento y desvíos con BFS.

9.4. Pregunta 4: ¿Por qué eligieron dichos algoritmos?

Porque el problema es, en el fondo, un problema de caminos mínimos en un grafo no ponderado y dirigido. BFS resuelve ese caso de forma exacta y con complejidad O(V + E), que es óptima para recorrer el grafo. A* resuelve el mismo problema con una heurística admisible y permite comparar el efecto de la información adicional sobre la cantidad de nodos expandidos. Los dos son algoritmos de la materia y se pueden explicar y justificar sin cajas negras.

Para las waves y el orden de paradas elegimos greedy porque la consigna no pide el óptimo global y prohíbe metaheurísticas y solvers. Un greedy determinista es fácil de reproducir, de medir y de comparar; además, permite aislar el efecto de una decisión (prioridad, distancia, capacidad) y entender por qué un escenario mejora o empeora. La alternativa exacta —probar todas las particiones y todos los órdenes— tendría un costo factorial inviable para 30 pedidos y 14 waves.

También elegimos estas herramientas porque el código tenía que quedar claro para la defensa oral. Preferimos un algoritmo simple que podamos explicar línea por línea antes que una optimización difícil de justificar. La comparación experimental muestra que, aun siendo simples, las estrategias logran mejoras importantes.

9.5. Pregunta 5: ¿Qué ocurre si todos los pasillos tienen el mismo largo?

El algoritmo no depende de que los pasillos tengan largos distintos. El grafo se construye celda por celda y la única información que importa es qué celdas son transitables y en qué dirección. Si todos los pasillos tienen el mismo largo, el warehouse tiende a ser más regular y las distancias dependen sobre todo de la cantidad de pasillos y de la posición relativa de los racks, no de la longitud individual. En un layout regular, BFS explora “en rombo” y A* con Manhattan se orienta muy bien; los empates de distancia son más frecuentes, pero se resuelven de forma determinista por orden de inserción y por ID.

Si los pasillos son todos del mismo largo y además todos AMBOS, el problema se parece a una grilla libre y la distancia entre dos celdas se acerca a la Manhattan. Si son de un solo sentido, la dirección pesa más que el largo: un pasillo corto en contra del sentido puede obligar a un rodeo mucho más largo que un pasillo largo a favor. En nuestros escenarios hay pasillos de largos distintos, pero el escenario básico y el de cercanía tienen layouts bastante regulares y se comportan como describe esta explicación.

No hicimos un experimento específico que aísle la variable “todos los pasillos iguales”; lo que sí hicimos fue comparar layouts regulares y con esquinas. Si el equipo quiere agregar una medición puntual, se puede generar un escenario con todos los pasillos de la misma longitud y correr las mismas cuatro combinaciones. La conclusión esperada es que no cambia la correctitud del algoritmo, solo la forma de los recorridos y la frecuencia de los empates.

9.6. Pregunta 6: ¿Cómo cambia el comportamiento del algoritmo cuando aumenta el tamaño del Warehouse?

El grafo crece con la cantidad de celdas transitables, y el costo de BFS y A* crece con V + E. En la práctica, el crecimiento es aproximadamente lineal en la cantidad de celdas, con constantes que dependen de la cantidad de vecinos. En los escenarios generados, pasar de 177 a 355 nodos hizo subir el tiempo medio de ruta de 0,23 a 0,74 ms; el tiempo de armado de waves subió de 12,5 a 32,5 ms. En el escenario grande, con 1.857 nodos, el tiempo medio por wave con cercanía + BFS fue 2,37 ms, y el armado de las 14 waves, 228 ms.

El crecimiento no es solo cuestión de nodos: también influyen la cantidad de paradas por wave, la cantidad de waves, la cantidad de pickers y la forma del layout. Un warehouse más grande pero con pasillos anchos y buen conectividad puede ser más fácil de recorrer que uno chico con muchos callejones. La tabla de distancias memoizada amortigua el crecimiento del armado: se corre un BFS por rack distinto, no por ítem. En el escenario grande, 68 BFS alcanzan para 14 waves.

El costo de construir el grafo de navegación también crece: 11,15 ms para el grafo completo y 34,98 ms para el de navegación en el escenario grande. Es un costo que se paga una vez por escenario. Si el warehouse creciera a decenas de miles de celdas, habría que pensar en construcciones incrementales o en precomputar distancias, porque la tabla actual guarda una fila por rack y puede ocupar bastante memoria.

9.7. Pregunta 7: ¿Cómo determinaron la distancia entre las ubicaciones de una wave?

La distancia entre dos ubicaciones se calcula como el camino mínimo en el grafo dirigido, con BFS o A*. Cada arista cuesta 1, así que la distancia es la cantidad de pasos del picker. Para un producto, la ubicación no es la celda del rack sino cualquiera de las celdas de pasillo ortogonalmente adyacentes desde las que se puede recoger. Por eso la distancia a un producto es la menor distancia a su conjunto de celdas de recogida.

Para armar waves no alcanza con la distancia entre dos productos: hace falta la distancia de la wave completa a cada candidato. La solución es una tabla de distancias por rack. La primera vez que se necesita un rack, se corre un BFS multi-origen desde sus celdas de recogida y se guarda la fila {celda: distancia}. La distancia de la wave a una celda es el mínimo entre las filas de sus racks. Al agregar un ítem, se actualiza ese mínimo celda por celda. Así se obtiene lo mismo que un BFS multi-origen desde todas las celdas de recogida de la wave, pero sin recalcular los racks que ya están en la tabla.

En el ruteo, la distancia entre paradas se calcula con `bfs_to_any` o `astar_to_any`, que buscan el camino mínimo hasta cualquiera de las celdas de recogida de la parada siguiente. En la estrategia de cercanía se hace una sola búsqueda hacia todas las paradas pendientes, y el primer destino que se toca es el más cercano. La altura z no se incluye en la distancia de camino; se suma aparte como costo de recogida, porque no cambia el recorrido sino el tiempo que el picker está quieto.

9.8. Pregunta 8: ¿Cómo decidieron qué pedidos pertenecen a una misma wave?

Con una estrategia greedy determinista que respeta las restricciones de capacidad y prioridad. Hay tres variantes. En `por_pedido`, se ordenan los pedidos por prioridad y se abre una wave con el más prioritario; después se agregan pedidos completos mientras entren, eligiendo por prioridad y, si empatan, por cercanía a la wave. En `por_cercania`, se trabaja ítem por ítem: la semilla es el ítem del pedido más prioritario y se agrega el ítem de mayor prioridad que esté más cerca, aunque sea de otro pedido. En `solo_cercania`, se ignora la prioridad y se agrega siempre el ítem más cercano.

Las tres respetan el máximo de pedidos distintos, el máximo de unidades y el peso máximo de la wave. El peso máximo se toma como la capacidad del picker operativo más grande, para que siempre haya alguien que pueda cargarla. Si un pedido no entra completo en una wave vacía, se divide: se agregan los ítems que caben y el resto queda para la siguiente. En las estrategias por ítem la división es más frecuente, pero permite agrupar productos cercanos de pedidos distintos.

La estrategia que quedó como opción por defecto en la aplicación es `por_cercania`, porque respeta la prioridad y además aprovecha la cercanía. `solo_cercania` obtiene mejor distancia en el escenario grande (2.167 contra 2.481), pero puede postergar pedidos urgentes; por eso se ofrece como alternativa de comparación. `por_pedido` es la más conservadora y la que menos divide pedidos, y sirve como línea base.

9.9. Pregunta 9: ¿Qué estrategia utilizaron para ordenar las ubicaciones de una wave?

Implementamos dos. La primera, `orden`, visita las paradas en el orden en que aparecen los pedidos dentro de la wave. La segunda, `cercania`, es un greedy de vecino más cercano: en cada paso elige la parada pendiente más cercana a la celda actual. La cercanía no hace una búsqueda por parada: hace un solo BFS hacia todas las celdas de recogida pendientes y corta en la primera que toca, que es la más cercana. Si varias paradas comparten la celda de llegada, se sirven todas juntas.

La cercanía tiene una regla adicional para los pasillos de un solo sentido: antes de elegir el próximo destino, descarta las celdas de recogida que obligarían a dejar atrás una parada pendiente. Para eso usa el progreso de la celda dentro del pasillo (y en N, −y en S, x en E, −x en O, 0 en AMBOS). Si una parada pendiente solo se puede recoger desde celdas con menor progreso en el mismo pasillo, la celda actual queda bloqueada y se sirve primero la otra. Esa regla evita rodeos largos y fue uno de los últimos ajustes del proyecto.

Las dos estrategias usan BFS o A* para cada tramo. La diferencia está en qué parada se elige, no en cómo se calcula el camino. Las métricas de cada recorrido permiten comparar ambas sobre la misma wave y con el mismo buscador.

9.10. Pregunta 10: ¿Qué diferencias existen entre las estrategias implementadas?

Hay diferencias en tres niveles. En el ruteo, `orden` conserva la secuencia de los pedidos y `cercania` reordena las paradas por distancia. En la generación de waves, `por_pedido` agrupa pedidos completos, `por_cercania` agrupa ítems de distintos pedidos respetando prioridad y `solo_cercania` agrupa ítems sin mirar prioridad. En el buscador, BFS y A* encuentran la misma distancia mínima, pero A* expande menos nodos y suele tardar más en tiempo de pared por el costo del heap.

La diferencia práctica más visible es la distancia. En el escenario grande, `cercania` recorre 2.481 pasos contra 3.751 de `orden` (−33,9 %). En el escenario de racks en esquinas, en cambio, `orden` recorre 26

contra 28 de `cercania`: el greedy se equivoca porque la parada “más cercana” en línea recta obliga a un rodeo por la geometría. Ese contraste es el mejor ejemplo de que las estrategias no son intercambiables.

En waves, `solo_cercania` obtiene 2.167 pasos, `por_cercania` 2.481 y `por_pedido` 2.904. La diferencia entre las dos primeras es el respeto por la prioridad; la diferencia entre `por_cercania` y `por_pedido` es que la primera puede dividir pedidos y agrupar ítems cercanos de pedidos distintos. En tiempo de armado las tres están cerca (222,8 a 235,5 ms), pero el ruteo es más barato cuando las waves quedan mejor agrupadas.

9.11. Pregunta 11: ¿Cuál presenta menor distancia recorrida?

La estrategia de cercanía presenta la menor distancia en la mayoría de los escenarios: gana en básico (52 contra 76), cuello de botella (81 contra 99), despacho múltiple (97 contra 121), niveles z (68 contra 92) y grande (2.481 contra 3.751). Empata en el escenario de cercanía (40 contra 40) y pierde en racks en esquinas (28 contra 26). Sumando los siete escenarios, la cercanía recorre 2.847 pasos contra 4.205 del orden, un 32,3 % menos.

En el escenario grande, la diferencia es de 1.270 pasos a favor de la cercanía (−33,9 %). Si se mira solo la generación de waves, `solo_cercania` es la que menos distancia total obtiene (2.167 pasos), seguida de `por_cercania` (2.481) y `por_pedido` (2.904). Es decir, la menor distancia global del experimento se logra combinando `solo_cercania` para armar waves y `cercania` para rutear.

La respuesta corta es: cercanía, con la salvedad de que no gana siempre. El contraejemplo de esquinas es parte de la respuesta, no una excepción que se pueda ignorar.

9.12. Pregunta 12: ¿Cuál presenta menor tiempo de ejecución?

En tiempo de pared, BFS suele ser más rápido que A* en nuestros escenarios. En el escenario grande, cercanía + BFS tarda 2,37 ms por wave contra 4,11 ms de cercanía + A*; orden + BFS tarda 4,71 ms contra 4,64 ms de orden + A*, una diferencia mínima. La razón es que el grafo no es lo bastante grande como para que el ahorro de nodos de A* compense el costo de mantener el heap y calcular la heurística en cada vecino. En un grafo mucho más grande, la relación podría invertirse.

Si se mira la cantidad de nodos expandidos, A* gana claramente: 9.262 contra 33.969 con cercanía y 18.117 contra 71.210 con orden. Esa métrica mide el trabajo del algoritmo, no el tiempo real. Para la consigna, las dos son útiles: los nodos expandidos explican por qué A* es una búsqueda informada, y el tiempo de pared muestra que en Python el costo constante importa.

La respuesta corta es: BFS para tiempo de ejecución en este prototipo; A* para cantidad de nodos expandidos. La estrategia de ruteo que menos tiempo total consume en el escenario grande es cercanía + BFS (33,22 ms para las 14 waves).

9.13. Pregunta 13: ¿Existe una estrategia que sea mejor en todos los escenarios?

No. El escenario de racks en esquinas es el contraejemplo más claro: `orden` recorre 26 pasos y `cercania` 28. La cercanía elige primero una parada que parece próxima, pero la geometría de las esquinas y los sentidos obligan a un rodeo; el orden, que no reordena, termina haciendo un recorrido más corto. En el escenario de cercanía hay empate, así que tampoco allí se puede decir que una sea estrictamente mejor.

Tampoco hay una estrategia mejor en todos los objetivos. `solo_cercania` gana en distancia, pero ignora la prioridad; `por_cercania` respeta la prioridad y queda en el medio; `por_pedido` conserva los pedidos completos y es más fácil de explicar. BFS gana en tiempo de pared, A* gana en nodos expandidos. La elección depende de qué se priorice y del layout del warehouse.

La conclusión es que conviene tener varias estrategias y compararlas. El sistema está pensado justamente para eso: la misma wave se puede correr con las cuatro combinaciones y ver las métricas. En un warehouse real, la estrategia se elegiría por layout y por objetivo operativo, no por una regla universal.

9.14. Pregunta 14: ¿Qué trade-offs encontraron entre tiempo de ejecución y calidad del recorrido?

- Distancia contra tiempo: la cercanía mejora la distancia, pero no siempre es más rápida de calcular. En el escenario grande, cercanía + BFS es a la vez más corta y más rápida que orden + BFS, pero en otros escenarios el greedy puede hacer más búsquedas o recorrer más nodos. La mejora de distancia no sale gratis en todos los casos.
- Nodos expandidos contra tiempo de pared: A* expande mucho menos, pero en Python el heap y la heurística tienen costo. Para este tamaño de grafo, BFS suele ganar en tiempo; A* gana en la métrica de trabajo del algoritmo.
- Prioridad contra distancia: `solo_cercania` obtiene la mejor distancia, pero puede dejar pedidos prioritarios para el final. `por_cercania` pierde algo de distancia y conserva la prioridad. Es un trade-off operativo, no solo algorítmico.
- Dividir pedidos contra conservar pedidos completos: las estrategias por ítem agrupan mejor y reducen distancia, pero complican el seguimiento de un pedido y pueden generar más waves con ítems de varios pedidos. `por_pedido` es más simple de explicar y de auditar.
- Más pickers contra coordinación: usar varios pickers reduce el tiempo de preparación, pero agrega colisiones, esperas, desvíos y lógica de estacionamiento. En pasillos angostos, el beneficio puede desaparecer si los pickers se bloquean.
- Memoria contra recálculo: la tabla de distancias acelera el armado de waves, pero guarda una fila por rack. En el escenario grande son 126.276 entradas; en un warehouse mucho más grande habría que evaluar si conviene guardar todo o recalcular algunas filas.

9.15. Pregunta 15: ¿Qué modificaciones realizarían para utilizar el sistema en un Warehouse real?

El sistema actual es un prototipo académico. Para llevarlo a un warehouse real habría que agregar, como mínimo: integración con el WMS/ERP para recibir pedidos y stock en tiempo real; persistencia en una base de datos en lugar de archivos JSON; posiciones reales de los pickers (RFID, lectores de código o ubicación indoor); y un motor de re-planificación que reaccione a pedidos nuevos, productos faltantes, pickers caídos o zonas llenas.

También habría que modelar mejor el tiempo. Hoy la recogida es un tick base más un tick por altura; en un warehouse real influyen la distancia al producto, la altura, el peso, el envase, la experiencia del picker y el tiempo de escaneo. La simulación tendría que calibrarse con datos históricos y probablemente usar distribuciones en lugar de valores fijos. Lo mismo pasa con las colisiones: en un warehouse real hay pasillos

más anchos, reglas de prioridad, zonas de espera y vehículos; el modelo de una celda por picker es una simplificación.

En escala, habría que pensar en un ruteo jerárquico o en precomputar distancias, en monitoreo y métricas en vivo, en seguridad (peso máximo, ergonomía, velocidad), en soporte para varios pisos y ascensores, y en una interfaz para supervisores. También haría falta validar el sistema con pruebas de aceptación y con usuarios reales, y definir qué hacer cuando el escenario es inviable: avisar, repartir la wave o cambiar la prioridad. Nada de esto está en el prototipo, y es importante decirlo: el objetivo del proyecto era entender los algoritmos, no construir un producto industrial.

10. Limitaciones y camino a un warehouse real

10.1. Limitaciones del prototipo

- El warehouse es estático: los racks, pasillos y productos no cambian durante la simulación. En un warehouse real, el stock cambia y los productos se reubican.
- El tiempo de recogida es una simplificación (1 tick base más 1 tick por altura). No modela escaneo, envase, peso, experiencia del picker ni tiempos estocásticos.
- Las estrategias son greedy y no garantizan el óptimo global. La comparación es experimental, no una demostración formal de optimalidad.
- La simulación de colisiones usa una celda por picker y reglas de prioridad. No modela pasillos anchos, vehículos, zonas de espera ni reglas de tránsito más complejas.
- Los escenarios son sintéticos y las mediciones se hicieron en una sola máquina. Los tiempos absolutos no se pueden extrapolar sin recalibrar.
- La persistencia es JSON y el servidor es stateless. No hay base de datos, usuarios, permisos ni auditoría.
- No hay integración con un WMS ni con lectores de código o RFID. La posición del picker es la que calcula la simulación.

10.2. Qué cambiaríamos para un uso real

Lo primero sería separar el motor de decisión del simulador. Hoy el mismo proceso calcula y dibuja; en producción convendría un servicio que reciba el estado del warehouse y devuelva planes, y otro que ejecute y monitoree. También habría que reemplazar los archivos JSON por una base de datos transaccional y agregar una capa de eventos para pedidos, stock y posiciones.

Lo segundo sería mejorar el modelo de costos. Un picker real no camina en una grilla perfecta: acelera, frena, busca el producto, escanea, acomoda la caja y a veces vuelve sobre sus pasos. Esos tiempos se pueden calibrar con datos históricos y usar para comparar estrategias con una función de costo más realista. La distancia en pasos seguiría siendo una métrica útil, pero no la única.

Lo tercero sería la re-planificación. En un warehouse real los pedidos llegan todo el tiempo y las condiciones cambian. El sistema tendría que poder reasignar waves, cambiar el orden de las paradas y mover pickers sin reiniciar la simulación. Eso abre problemas de consistencia y de comunicación que el prototipo no aborda. También habría que definir políticas para escenarios inviables: avisar al supervisor, partir la wave, priorizar otro pedido o dejar el pedido pendiente.

Por último, habría que trabajar en la interfaz para supervisores: mapa en vivo, alertas, métricas por turno, comparación de estrategias y trazabilidad de cada pedido. La visualización actual sirve para entender los algoritmos; una herramienta de operación necesita otra información y otras decisiones.

10.3. Alcance declarado

El proyecto cumple el objetivo de modelar el problema y aplicar algoritmos de grafos, estructuras de datos y análisis de complejidad. No pretende ser un sistema industrial. Las simplificaciones están declaradas y son razonables para el alcance de la materia. La defensa del trabajo se apoya en que las decisiones están justificadas, el código es modular, los tests pasan y los experimentos se pueden reproducir.

11. Conclusiones

11.1. Conclusiones técnicas

El sistema logra representar el warehouse, construir un grafo dirigido que respeta los sentidos de circulación, generar waves con restricciones de capacidad y prioridad, calcular recorridos con dos estrategias y dos buscadores, y simular la preparación con varios pickers. La separación entre modelo, algoritmos, simulación e interfaz permitió probar cada parte por separado y mantener la lógica fuera del frontend.

Los experimentos muestran que la cercanía mejora la distancia en la mayoría de los escenarios, pero no en todos. El caso de racks en esquinas es el recordatorio de que un greedy no es una garantía: la parada más cercana en línea recta puede obligar a un rodeo. BFS fue más rápido en tiempo de pared en casi todos los casos, mientras que A* expandió muchos menos nodos. Esa diferencia entre “trabajo del algoritmo” y “tiempo real” fue uno de los aprendizajes más interesantes del proyecto.

La generación de waves tuvo tanto impacto como el ruteo. Pasar de pedidos completos a ítems cercanos redujo la distancia, a costa de dividir pedidos y de un tiempo de armado algo mayor. La tabla de distancias memoizada fue clave para que ese costo no creciera de forma innecesaria. La regla de no dejar paradas atrás en pasillos de un solo sentido resolvió un problema que no se ve en un layout ideal pero aparece en cuanto hay direcciones.

El balance general es positivo: el sistema es explicable, medible y reproducible. Las limitaciones están claras y el camino a un warehouse real está identificado. No encontramos una estrategia universal, y creemos que esa es una conclusión honesta: el valor está en poder comparar y elegir con datos, no en vender una solución mágica.

11.2. Qué nos llevamos

Aprendimos a pensar el problema en capas: primero el modelo físico, después el grafo, después los algoritmos y al final la visualización. También aprendimos que la complejidad teórica no alcanza: dos algoritmos con la misma solución pueden tener costos muy distintos en la práctica, y hay que medir. La memoización, el uso de `deque` y `heapq`, y el cuidado con los casos borde (pasillos sin retorno, pickers bloqueados, zonas llenas) fueron aprendizajes concretos que nos llevamos de la materia.

Otro aprendizaje fue la importancia de los tests. Los 110 tests y 61 subtests no cubren todos los escenarios posibles, pero nos dieron confianza para cambiar el grafo, el ruteo y la simulación sin romper lo anterior.

También nos obligaron a escribir mensajes de error claros, que después sirvieron para diagnosticar escenarios inviables.

12. Referencias

12.1. Bibliografía y documentación

- Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. Introduction to Algorithms. MIT Press. Capítulos de grafos, BFS, A* y componentes fuertemente conexas.
- Python Software Foundation. Documentación de `collections.deque` y `heapq`. https://docs.python.org/3/library/collections.html y https://docs.python.org/3/library/heapq.html
- Pallets Projects. Documentación de Flask. https://flask.palletsprojects.com/
- Mozilla Developer Network. Canvas API. https://developer.mozilla.org/es/docs/Web/API/Canvas_API
- Repositorio del proyecto: https://github.com/gracimarch/proyecto-final-aed2
- Cátedra de Algoritmos y Estructuras de Datos II, Universidad Nacional de Cuyo. Apuntes y material de clase.

12.2. Uso de herramientas

Durante el desarrollo usamos Git y GitHub para el control de versiones, Python y Flask para el backend, HTML/CSS/JavaScript para el frontend y `pytest` para las pruebas. Para el análisis experimental usamos `time.perf_counter()` de la biblioteca estándar y `matplotlib` para los gráficos de este informe. Las figuras de resultados se generaron a partir de las mediciones propias.

13. Anexos

Anexo A. Reproducción de la aplicación y de las mediciones

La aplicación se ejecuta con Flask y no necesita una base de datos. Los pasos para levantarla en local son los del README del repositorio:

```
git clone https://github.com/gracimarch/proyecto-final-aed2.git
cd proyecto-final-aed2
python -m venv venv
venv\Scripts\activate          # Windows
pip install -r requirements.txt
python app.py
# abrir http://127.0.0.1:5000/
```

Para correr las pruebas:

```
python -m pytest -q
# resultado al momento de este informe:
# 110 passed, 61 subtests passed
```

Las mediciones de este informe se hicieron con un script que recorre los escenarios, construye el warehouse y el WaveManager, genera las waves y ejecuta las cuatro combinaciones de estrategia y buscador. Cada medición se repite cinco veces y se guarda la mediana. El esquema es el siguiente:

```
for escenario in escenarios:
    datos = json.load(open(escenario, encoding="utf-8"))
    wh = Warehouse.from_dict(datos)
    wm = WaveManager(warehouse=wh)
    wm.add_orders([Order.from_dict(o, wh.products) for o in datos["pedidos"]])
    wm.generate_waves(estrategia_wave="por_cercania")

    for estrategia in ("orden", "cercania"):
        for buscador in ("bfs", "astar"):
            tiempos = []
            for _ in range(5):
                t0 = time.perf_counter()
                for wave in wm.get_waves():
                    plan_route(wave, wh, wm.graph, estrategia, buscador)
                tiempos.append((time.perf_counter() - t0) * 1000)
            mediana = statistics.median(tiempos)
```

El script exacto y los resultados crudos quedaron fuera del repositorio para no mezclar documentación con código. Si se quiere volver a medir, alcanza con reconstruir el bucle anterior usando las funciones públicas del proyecto. Las figuras de este informe se generaron con `matplotlib` a partir de esos resultados.

Anexo B. Estructura del JSON de un escenario

Un escenario se guarda en un archivo JSON con las claves que se resumen en la Tabla 11. La aplicación valida el archivo antes de cargarlo y devuelve un error concreto si algo no cumple.

Tabla 11. Claves principales del JSON de escenario.

Clave	Contenido
warehouse.dimensiones	ancho_x y largo_y del warehouse.
warehouse.puertas	Lista de puertas con id, tipo y ubicación sobre el borde.
warehouse.zonas_despacho	Rectángulos frente a una puerta, con peso_max_kg.
warehouse.racks	Rectángulos con un lado igual a 2.
warehouse.pasillos	Rectángulos con sentido N, S, E, O o AMBOS.
pickers	id, posición inicial y capacidad_carga_kg.
productos	id, nombre, rack_id, ubicación (x, y, z), stock y peso_kg.
pedidos	id, prioridad (1 a 5), estado e ítems (producto_id, cantidad).
configuracion	wave_max_pedidos y wave_max_productos.